Building a Custom BPE Tokenizer with WikiText-2

A Practical Exploration of Subword Tokenization for Modern NLP Systems

---

Executive Summary

This project develops a custom Byte Pair Encoding (BPE) tokenizer using the WikiText-2 dataset. The objective is to understand how modern Natural Language Processing (NLP) systems convert raw text into reusable subword units while maintaining vocabulary efficiency and language coverage.

The project includes dataset exploration, corpus cleaning, tokenizer training, vocabulary analysis, evaluation, and deployment of an interactive tokenizer visualization. A 30,000-token vocabulary was successfully learned from a cleaned corpus of 21,337 documents. Evaluation demonstrated strong compression efficiency and effective handling of previously unseen words through subword decomposition.

---

1. Introduction

1.1 Background

Large Language Models (LLMs) such as GPT, BERT, Gemma, and LLaMA do not process raw text directly. Instead, text is transformed into tokens before being converted into numerical representations.

A tokenizer therefore serves as the first stage of every modern NLP pipeline.

Traditional word-level tokenization suffers from several limitations:

- Large vocabulary requirements
- Poor handling of unseen words
- Vocabulary sparsity
- Inefficient memory usage

Subword tokenization techniques such as Byte Pair Encoding (BPE) address these challenges by learning reusable subword units from data.

1.2 Project Objectives

The objectives of this project are:

- Load and explore the WikiText-2 dataset
- Perform corpus cleaning and deduplication
- Train a custom BPE tokenizer
- Learn a vocabulary of 30,000 tokens
- Analyze learned vocabulary structures
- Evaluate tokenizer efficiency
- Save and reload the tokenizer
- Deploy an interactive visualization

---

2. Understanding Byte Pair Encoding

2.1 What is BPE?

Byte Pair Encoding (BPE) is a data compression-inspired tokenization algorithm that iteratively merges the most frequently occurring adjacent symbol pairs.

The algorithm begins with individual characters and gradually learns larger linguistic units such as:

- Character pairs
- Subwords
- Complete words

This approach enables compact vocabularies while maintaining coverage of rare and unseen words.

2.2 Why BPE?

Advantages include:

- Reduced vocabulary size
- Better handling of rare words
- Improved language coverage
- Lower out-of-vocabulary rates
- Efficient sequence representation

---

3. Dataset Overview

3.1 WikiText-2 Dataset

The WikiText-2 dataset is a widely used benchmark corpus derived from Wikipedia articles.

Dataset Characteristics:

- Source: WikiText-2
- Total Rows: 44,836
- Training Rows: 36,718
- Validation Rows: 3,760
- Test Rows: 4,358

---

4. Data Cleaning and Preprocessing

4.1 Cleaning Strategy

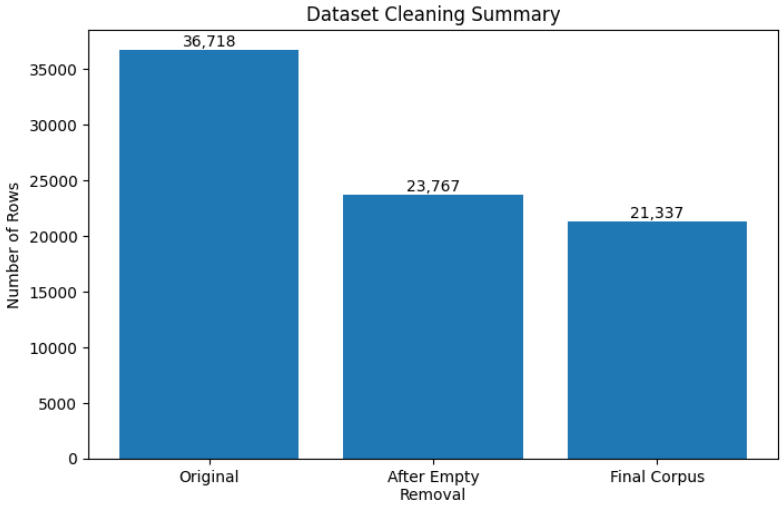
The following preprocessing steps were applied:

1. Remove empty rows
2. Remove duplicate entries
3. Remove <unk> placeholders
4. Normalize whitespace

4.2 Cleaning Results

Stage	Rows
Original Training Rows	36,718
Empty Rows Removed	12,951
Duplicate Rows Removed	2,430
Final Corpus	21,337

Figure 1: Dataset Cleaning Summary



---

5. Corpus Exploration

5.1 Vocabulary Statistics

The cleaned corpus contains:

- 21,337 documents
- 33,276 unique words

This vocabulary size motivates the need for subword tokenization rather than pure word-level approaches.

---

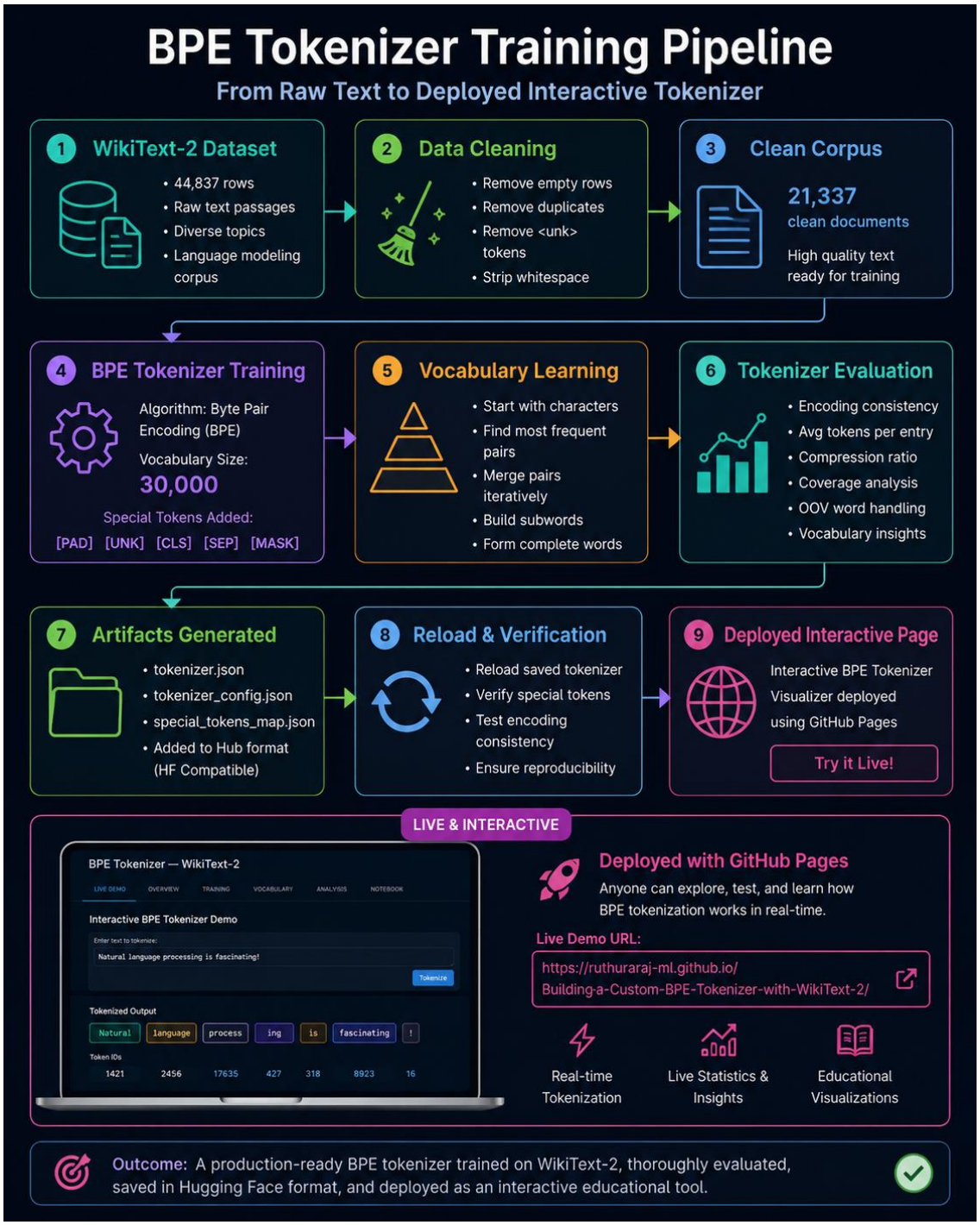
6. BPE Tokenizer Design

6.1 Training Configuration

Parameter	Value
Algorithm	BPE
Vocabulary Size	30,000
Pre-tokenizer	Whitespace
Special Tokens	[PAD], [UNK], [CLS], [SEP], [MASK]

6.2 Architecture Pipeline

Figure 2: BPE Tokenizer Training Pipeline



7. Tokenizer Training

7.1 Training Process

The tokenizer was trained using Hugging Face's Tokenizers library.

Training involved:

- Character initialization
- Frequency counting
- Pair merging
- Vocabulary construction

until the target vocabulary size was reached.

---

8. Vocabulary Analysis

8.1 Learned Vocabulary Structure

Low-level tokens:

- th
- in
- er
- ed

Mid-level tokens:

- state
- million
- design

Advanced tokens:

- potassium
- sarcophagus
- Ferguson

This demonstrates the hierarchical nature of BPE learning.

---

9. Rare Word Analysis

9.1 Generalization Capability

Examples:

Word	Decomposition
internationalization	international + ization
transformational	transform + ational
ChatGPT	Ch + at + GP + T
Ruthuraraj	Ru + thur + ara + j

The tokenizer successfully represents previously unseen words through reusable subword units.

Figure 3: Rare Word Decomposition Examples

```
internationalization
→ international + ization

transformational
→ transform + ational

someone@somemail.com
→ someone + @ + som + email + . + com

Ruthuraraj
→ Ru + thur + ara + j

ChatGPT
→ Ch + at + GP + T
```

---

10. Evaluation

10.1 Quantitative Results

Metric	Value
Vocabulary Size	30,000
Average Tokens per Entry	84.96
Compression Ratio	4.97
Tokenization Consistency	Deterministic

10.2 Interpretation

A compression ratio of 4.97 indicates that each token represents nearly five characters of information on average.

This demonstrates efficient language representation while preserving vocabulary coverage.

---

11. Interactive Deployment

The trained tokenizer was deployed as an interactive educational application using GitHub Pages.

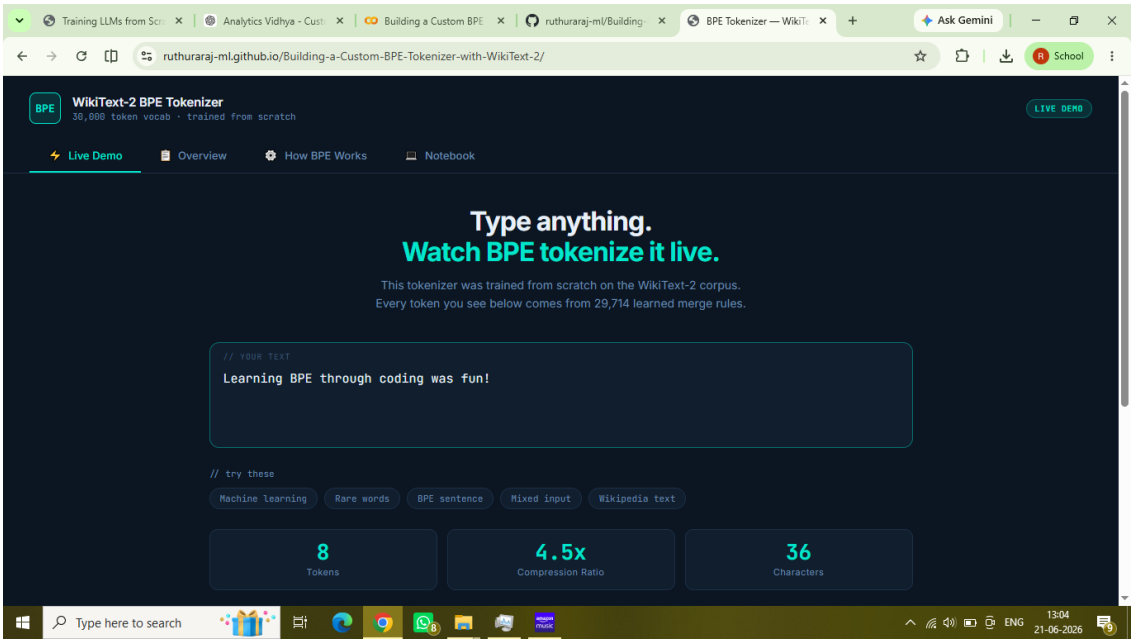
Features include:

- Live tokenization
- Vocabulary exploration
- Educational explanations
- Interactive demonstrations

Deployment URL:

https://ruthuraraj-ml.github.io/Building-a-Custom-BPE-Tokenizer-with-WikiText-2

Figure 4: Interactive BPE Tokenizer Visualizer



12. Key Findings

- BPE successfully learned a 30,000-token vocabulary.
- The tokenizer generalized effectively to unseen words.
- Compression efficiency approached 5:1.
- Vocabulary evolved from characters to meaningful subwords and words.
- Interactive deployment improved accessibility and educational value.

---

13. Conclusion

This project demonstrated the complete lifecycle of building a custom BPE tokenizer using WikiText-2. Through cleaning, training, evaluation, and deployment, the project highlighted why subword tokenization remains a foundational component of modern NLP systems.

The results show that a relatively compact vocabulary can efficiently represent both common and previously unseen language constructs through reusable subword units.

References

1. Hugging Face Tokenizers Documentation
2. Hugging Face Datasets Documentation
3. WikiText-2 Dataset
4. Sennrich et al. (2016), Neural Machine Translation of Rare Words with Subword Units